Process-to-Process Communication, DNS, and HTTP Fundamentals

Introduction to Web Technology — Lecture 2

Index

- Process-to-Process Communication over the Internet — sockets: the doors your packets sneak out of.
- What a Web Browser Actually Does — more than a picture-fetcher, it's a tiny runtime.
- A Brief History of the Web — how a CERN paperwork problem became a trillion-dollar industry.
- The Web Evolved in Three (and a Half) Waves — from read-only pages to machines gossiping with each other.
- Static vs. Dynamic Web Pages — the difference between a poster and a live conversation.
- DNS: The Internet's Phone Book — because nobody memorizes IP addresses for fun.
- Resolving a Domain Takes Several Hops — recursive vs. iterative, or "who does the walking?"
- The DNS Name Space — an upside-down tree where "com" is basically royalty.
- Types of DNS Servers — root, TLD, and authoritative — the three-tier bureaucracy of names.
- Web and HTTP: Anatomy of a URL — dissecting that string you never actually read.
- HTTP Is a Request–Response Protocol — the internet's most polite conversation format.
- More on HTTP: TCP, Ports, and Statelessness — HTTP has amnesia, and that's by design.
- Anatomy of HTTP Messages — request and response, line by line.
- Status Codes Summarise the Outcome in One Number — the internet's way of grunting "yes," "no," or "not my problem."
- GET vs. POST — asking politely vs. handing over a form.
- Practical Questions to Chew On

Process-to-Process Communication over the Internet

The web follows a **client–server architecture**. The client is your web browser; the server is some process on another host holding data or logic the browser wants. Since it is *processes* — not machines — that actually exchange messages, we need a way to address a process specifically, not just the machine it runs on.

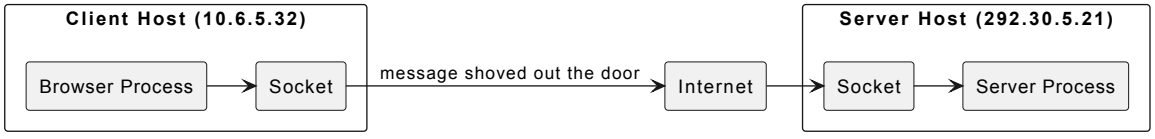
This is done through a **socket**. A socket is best understood by analogy: it is a *door*. The sending process shoves its message out through this door and then relies entirely on the transport infrastructure on the other side to deliver it to the receiving process's own door (its socket). Every communicating pair therefore involves two sockets — one at each end — and together they behave like a point-to-point pipe between two processes, even though the underlying network path may hop across many routers.

Two different identifiers are at play here, and it's easy to conflate them:

- **IP address** — operates at the *network layer*. It uniquely identifies a *host* (machine) on a network, e.g. `10.6.5.32` or `192.30.5.21`. Routers use only this to move packets closer to their destination.
- **Port number** — operates at the *transport layer*. A single host can run many processes/applications simultaneously (a mail server, a web server, a chat client, etc.), so once a packet arrives at the correct IP address, something is still needed to hand it to the *correct process*. That "something" is the port.

$$\text{Socket} = (\text{IP address} : \text{Port number})$$

So the socket is the actual transport-layer endpoint that uniquely identifies one application instance on one host — written conventionally as `IP:Port`.



Well-known port examples:

Service	Port
HTTP server	80
Mail server	25

So, to send an HTTP message to the web server at `gaia.cs.umass.edu`, a client needs *both* pieces of the identifier: the IP address (`128.119.245.12`) and the port number (`80`).

> **Note for Review:** Students often mix up "IP address" and "socket." Remember: IP identifies the *host*; socket identifies the *process on that host*. Without the port, a packet could reach the right building but never find the right office.

What a Web Browser Actually Does

A browser is not a passive display surface — it is a small client-side runtime with several distinct jobs:

- **Send requests, receive responses** — over HTTP/HTTPS.
- **Parse HTML/CSS/JS and render** — turning marked-up text into the visual page you see.
- **Run a JavaScript engine** — for interactivity and dynamic behaviour; some computation happens *inside* the browser itself, not just data fetching.
- **Manage cookies, cache & security** — same-origin policy, the HTTPS padlock, and remembering state across requests.
- **Common engines:** Blink (used by Chrome, Edge — descended from WebKit), WebKit (Safari), Gecko (Firefox).

Because different browsers use different rendering engines, the *same* web content can be interpreted and displayed slightly differently across platforms — this is why cross-browser testing matters.

Two performance-related concerns come up repeatedly at this layer:

- **Sessions** — session-based communication (post-authentication) lets a server treat a sequence of requests as belonging to one continuous interaction, which is important for security.
- **Caching** — storing commonly visited pages/resources locally in the browser to avoid refetching them, improving perceived performance.

A Brief History of the Web

1989 — CERN (European Centre for Nuclear Research). Large research teams needed to collaborate using a constantly changing collection of reports, blueprints, drawings, photos, and other documents, while ensuring every client saw an up-to-date version. The proposal for a solution — the hypertext concept — came from **Tim Berners-Lee**. It was publicly demonstrated at the **Hypertext '91** conference.

1993 — The first graphical browser, Mosaic, was developed by **Marc Andreessen** at the University of Illinois. Andreessen went on to form **Netscape Communications Corp**. Microsoft responded with Internet Explorer, sparking the famous **"browser war"** between Internet Explorer and Netscape Navigator.

1994 — CERN and MIT signed an agreement to establish the **World Wide Web Consortium (W3C)**, which became (and remains) the standardization authority for the web. Once a technology is standardized by the W3C, it becomes part of the globally acknowledged web stack.

The Web Evolved in Three (and a Half) Waves

The web's growth is often described along an axis of *value created over time* (see this overview for the underlying chart), moving through four broad orientations:

Information-centered → People-centered → Machine-centered → Agent-centered

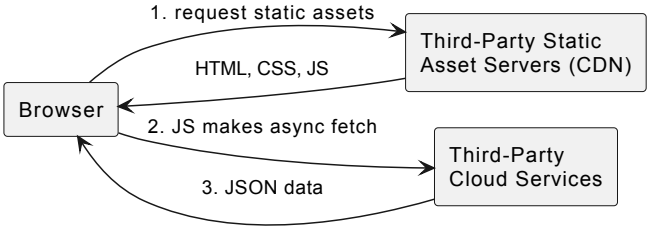
Era	Focus	Key Traits	Examples
Web 1.0	Information-centered	Fixed content, no user interaction, same for every visitor, read-only	Early e-commerce, desktop browser access
Web 2.0	People-centered	Comments/likes, content updates dynamically, server generates HTML per request, read-write	Facebook, Netscape-era social platforms
Web 3.0	Machine-centered	Real-time sync, APIs + microservices, decentralised data architecture, edge computing, read-write-execute	Uber, Airbnb, Bitcoin/Ethereum-style services
Web (next)	Agent-centered	AI-driven, autonomous agents interacting on your behalf	Emerging — "?" on the value curve

The market-cap figures cited alongside this curve were roughly **$1.1 trillion** for internet companies around 1990–2000, growing to about **$5.9 trillion** by the Web 2.0 era, with the trend continuing sharply upward toward 2025.

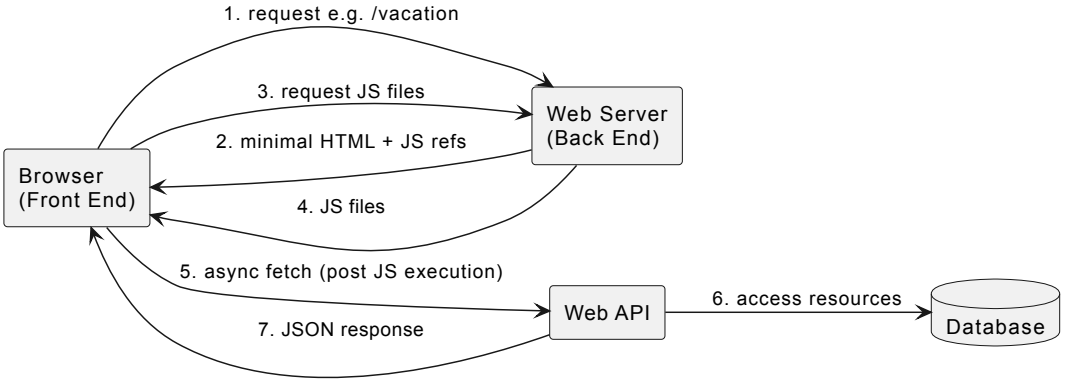
Concretely, this shows up as an evolution from **static HTML pages** (Web 1.0: fixed content, no interaction) → **blogs/CMS platforms** (Web 2.0: comments, dynamically updated content, server renders HTML per request) → **cloud-native web apps** (Web 3.0+: real-time sync, personalised, scale on demand via APIs and microservices).

Static vs. Dynamic Web Pages

Static path: The browser requests static assets (HTML, CSS, JS files) from a **third-party static asset server**, which is typically distributed worldwide (a CDN-like setup, deliberately bypassing dependence on a single global path). JavaScript in the returned page then makes asynchronous fetches to a **third-party cloud service**, which may itself talk to databases/other cloud services, returning data as JSON that JavaScript uses to render the final page.



Dynamic path: A request is made from the browser (essentially a blank placeholder waiting to be filled). The server performs **minimal processing** and returns HTML plus references to JavaScript files. The browser then separately requests those JS files; once they execute, they make further asynchronous fetches to a **Web API** (which may live on the same server or an entirely different one). The Web API accesses server-side resources such as databases and returns data in JSON, which JavaScript uses to continually update the page based on both the fetched data and user actions.



The essential design goal of a dynamic architecture is that the **back end can be arbitrarily complex — multiple round trips, data spread across multiple servers — but the user should never feel that they're talking to a remote server at all.** All of that complexity has to be hidden behind a seamless-feeling front end.

DNS: The Internet's Phone Book

Humans use names, machines use addresses. Nobody wants to memorize `93.184.216.34` when `example.com` will do — humans are good at remembering names, machines are good at handling numbers. **DNS (Domain Name System)** exists purely to bridge this gap: given a domain name, it returns the corresponding IP address. This translation happens at your nearest domain name server.

Key properties:

- **Distributed & hierarchical** — no single server holds every record. Given the sheer number of domains worldwide, keeping one central record simply isn't feasible; hierarchy is what makes it scalable.
- **Cached at multiple levels** — browser, OS, resolver — so repeated lookups don't have to walk the whole hierarchy again. This is the same caching principle as browser asset caching, applied to name resolution.
- **Resolves host name conflicts** — a naming hierarchy needs to be actively managed. This is rooted at **ICANN** (Internet Corporation for Assigned Names and Numbers), the authority governing how names are registered and organized globally.

DNS Services:

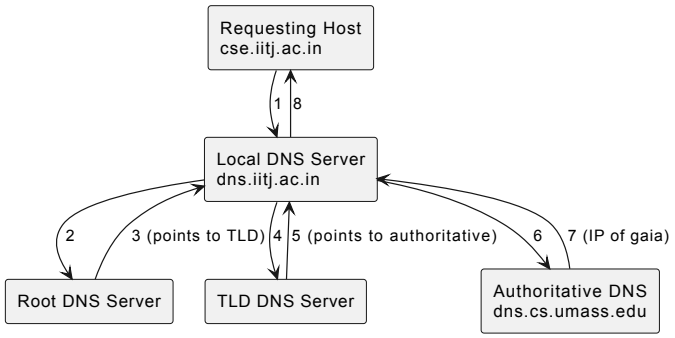
- Hostname-to-IP-address translation
- Host aliasing (canonical and alias names — one IP can answer to multiple names)
- Mail server aliasing (multiple distinct mail servers presented under one common name, for scaling)
- Load distribution — a **set of IP addresses mapped to one canonical name**, so a replicated web server (e.g., `google.co.in`) can silently route you to any of several backend machines.

Resolving a Domain Takes Several Hops

Suppose a host at `cse.iitj.ac.in` wants the IP address for `gaia.cs.umass.edu`. Since every network has its own **local DNS server**, the very first thing that happens — before *any* packet can be routed, since routers only understand IP addresses — is name resolution via this local server.

There are two resolution strategies:

Iterative resolution — the local DNS server does all the walking itself, querying each level and coming back each time:



At each step, a higher-level server doesn't know the final answer but knows *who to ask next* — the root points to the TLD server, the TLD server points to the authoritative server for that specific domain.

Recursive resolution — each server passes the query fully onward and waits, rather than the local server managing each hop itself:



Once resolved, the host can finally address its packets with the correct destination IP (e.g. `178.16.5.20`).

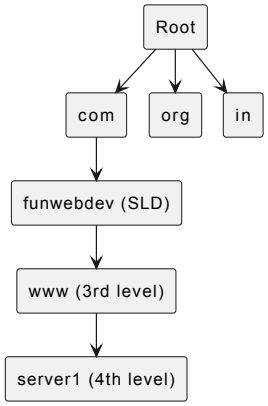
> **Note for Review:** Recursive vs. iterative is a classic exam trade-off. **Iterative** keeps the local DNS server *more responsive/observable* (you can watch progress at each hop) but the query takes a longer, more roundabout route. **Recursive** puts more work — and more waiting — onto the upper-level servers (root/TLD), but it *reduces load on the local DNS server*, since it just forwards once and waits, freeing it to service other requests in the meantime.

The DNS Name Space

Domain names form a **tree**, read right-to-left from most general to most specific:

```
server1 . www . funwebdev . com
 (4th)   (3rd)    (2nd)     (TLD)
```

- **Top-Level Domain (TLD)** — e.g. `com`, `org`, `in`, or country-specific codes like `uk`, `jp`.
- **Second-Level Domain (SLD)** — e.g. `funwebdev`.
- **Third-level domain** — e.g. `www`.
- **Fourth-level domain** — e.g. `server1`, typically the specific host/server.



Elements of DNS (RFC 1034):

- **Domain Name Space & Resource Records** — specifications for a tree-structured namespace and the data associated with each name.
- **Name Servers** — programs holding information about the domain tree's structure and set information. A particular name server has *complete* information about only a *subset* of the domain space; for that subset it is said to be an **authority**.
- **Resolvers** — programs that extract information from name servers in response to client requests.

Top-level domains are run by **registrars appointed by ICANN**. For India's `.in` domain specifically, the registrar is **registry.in**, managed by **NIXI** (National Internet Exchange of India).

Types of DNS Servers

- **Root Name Servers** — contacted by a local name server that cannot resolve a name itself. They contact the authoritative name server if the mapping isn't already known, and return the mapping back to the local server. There are **13 root name servers worldwide**, geographically distributed (operated by entities like Verisign, ICANN, RIPE, WIDE, etc.).
- **Top-Level Domain (TLD) servers** — responsible for `com`, `org`, `net`, `edu`, and country-specific TLDs like `.uk`, `.fr`, `.ca`, `.jp`. (Network Solutions, for example, maintains the servers for the `.com` TLD.)
- **Authoritative DNS servers** — an organization's own DNS servers, providing the authoritative hostname-to-IP mappings for that organization's servers (web, mail, etc.). These can be maintained either by the organization itself or by a third-party service provider.

Web and HTTP: Anatomy of a URL

A web page consists of **objects** — an HTML file, a JPEG image, a Java applet, an audio file, and so on. The page itself is a **base HTML file** that acts as a template referencing several such objects. Each object is individually addressable by a **URL (Uniform Resource Locator)**.

```
https://www.example.com:443/products/shoes?color=red&size=9#reviews
```

Component	Value	Meaning
Scheme/protocol	`https://`	`http` or its secure variant `https`
Host (domain)	`www.example.com`	The name standing in for the server's IP address
Port (optional)	`443`	The transport-layer endpoint on the host
Path	`/products/shoes`	Directory/sub-directory location of the resource on the server
Query string	`?color=red&size=9`	Additional formatting/filtering info about the object requested
Fragment	`#reviews`	A pointer to a specific section within the fetched resource

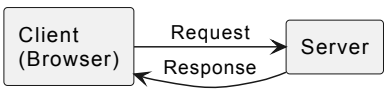
HTTP Is a Request–Response Protocol

Hypertext is a way to represent web content — text along with *formatting instructions* (the "metadata" describing how content should be presented, analogous to a teacher's markup on an exam paper: not the answer itself, but instructions about how to treat it).

HTML (Hypertext Markup Language) is the scripting language used to specify this web data along with simple formatting (bold, italics, new lines) — in essence, a way to convert text-based information into graphics-based, visually organized information. Today, much more (CSS for styling, JavaScript for behaviour) is embedded inside HTML, but HTML itself remains the irreplaceable base layer.

```
<!DOCTYPE html>
<html>
<!-- created 2010-01-01 -->
<head>
 <title>sample</title>
</head>
<body>
 <p>Voluptatem accusantium totam rem aperiam.</p>
</body>
</html>
```

The client (browser) requests this page; the server responds with its content:



This same request–response pattern works regardless of what device is running the browser — laptop, desktop, workstation, or phone — the underlying exchange is always an HTTP request out and an HTTP response back.

More on HTTP: TCP, Ports, and Statelessness

HTTP runs on top of TCP (Transmission Control Protocol):

1. The client **initiates a TCP connection** (i.e., creates a socket) to the server, conventionally at **port 80** — the well-known port for HTTP. (A custom application could instead pick a port from the user-defined range, but standard protocols always use their well-known port.)
2. The server **accepts** the TCP connection from the client.
3. **HTTP messages** (application-layer protocol messages) are exchanged between the browser (HTTP client) and the web server (HTTP server).
4. The **TCP connection is closed**.

Because TCP is a reliable, connection-oriented protocol requiring a three-way handshake, the connection persists for the duration of the exchange, giving HTTP a dependable transport substrate to sit on.

HTTP is "stateless" — the server retains **no memory** of past client requests. Each request is handled independently, as if it were the very first one. (State across requests, when needed, is later handled via **cookies**.)

> **Note for Review:** "Stateless" doesn't mean HTTP is *broken* by design — it's a deliberate simplicity trade-off. Because the server never has to track ongoing per-client context, it scales far more easily to millions of simultaneous, independent requests. The cost is that anything session-like (logins, shopping carts) has to be bolted on externally, typically via cookies.

Anatomy of HTTP Messages

Both HTTP request and response messages are written in **ASCII (human-readable format)**.

Request message structure:

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

- **Request line** — the method (`GET` , `POST` , `HEAD` , …), the requested resource, and the HTTP version.
- **Header lines** — host, user agent (client browser identity — lets the server customize responses), accepted content types, languages, encodings, and connection persistence settings.
- **`\r\n`** (carriage return + line feed) at the start of a line marks the **end of the header block**.

Response message structure:

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

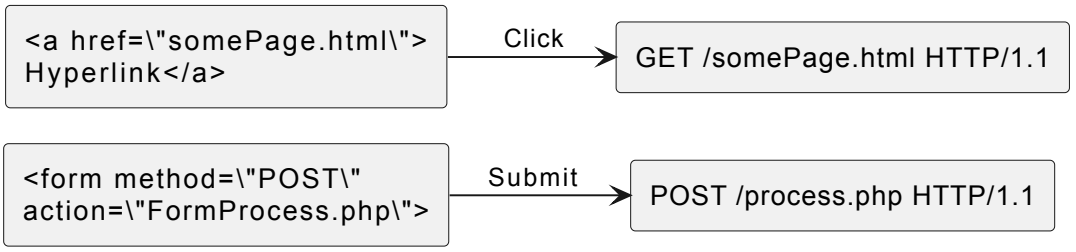
- **Status line** — protocol version, status code, and status phrase.
- **Header lines** — server details, last-modified timestamp, content type/length, caching-related tags.
- **Data** — the actual requested content (e.g. the HTML file itself), following the blank line after headers.

Status Codes Summarise the Outcome in One Number

Class	Meaning	Examples
2xx	Success	`200 OK` , `201 Created`
3xx	Redirection	`301 Moved Permanently` , `304 Not Modified`
4xx	Client error	`400 Bad Request` , `401 Unauthorized` , `404 Not Found`
5xx	Server error	`500 Internal Server Error` , `503 Service Unavailable` , `505 HTTP Version Not Supported`

Each numeric family communicates *whose fault it is and roughly what kind of problem occurred*, without the client needing to parse free text.

GET vs. POST



- **GET** — used for *fetching* a resource (e.g. clicking a hyperlink). Parameters typically travel in the URL itself.
- **POST** — used for *pushing* information to the server (e.g. submitting a form), where the payload travels in the request body rather than the URL.

Rule of thumb: GET requests should **not** be used for state-changing actions (such as modifying server-side data) — GET is meant to be a safe, idempotent read.

Practical Questions to Chew On

1. A single organization's web server sits behind one public IP address but must simultaneously run a mail server, an HTTPS website, and an internal SSH service for admins. Explain, in terms of sockets, exactly what changes (and what stays the same) as three completely different clients on the internet talk to this one machine at the same time.

2. Suppose a company deliberately configures its authoritative DNS server to return a *different* IP address depending on which geographic region the query came from (all under the same domain name). Which DNS service does this exploit, and what real-world problem is it solving that a single static A-record couldn't?

3. If HTTP is stateless, yet you stay "logged in" to a website as you click from page to page, walk through what mechanism must be compensating for that statelessness, and explain why that mechanism has to live outside the core HTTP request-response cycle rather than inside it.

4. A user complains that a page "hangs" for a few seconds before anything appears, but once it appears, scrolling and interacting with it feels instant. Based on the static-vs-dynamic web page architecture discussed, propose two different explanations (one DNS/network-related, one architecture-related) for where that initial delay could be coming from, and how you'd distinguish between them.

5. Compare the recursive and iterative DNS resolution approaches from the perspective of a root DNS server operator responsible for handling billions of queries a day. Which approach would you prefer root servers enforce, and why might that preference be different from the preference of the local DNS server administrator at a small college?